Lab Thirteen-ID1063

File Handling

1. Write recursive C functions to compute each of these mathematical functions.

   (a) The Tribonacci series is given by $T_0 = 0, T_1 = T_2 = 1$ and $T_n = T_{n-1} + T_{n-2} + Tn - 3$.

   (b) $t_0 = 1$ and $t_n = 2 + \frac{1}{t_{n-1}}$ for $n \geq 0$;

2. Write a program to read the file "1000CommonWords.txt" and print all the 7-letter words in this file on the screen.

3. This sample code (printDigitsRecursive.c in the shared folder) prints the digits of a number from right-to-left recursively.

   (a) Draw the recursion tree for the number 1234.

   (b) How should the tree look like if you wanted to print the digits from left to right.

   (c) Modify the program to print the digits from left to right.

4. (a) Read the program called findElement.c.

   (b) Write a program that accepts the name of a country from the user and prints its capital. Use the file capitals.txt for this.

   (c) Modify the findElement program as follows: create a structure called Element (use/modify the one from the previous lab), and create an array of Element variables. Read each line of the chemicalElements.txt file and store it into the Element array.

   (d) Write a function that takes an Element variable and prints its data. Use it on each of the array entries.

5. The sample code binaryStringsRecursive.c prints all binary strengths of a given length n.

   (a) Modify it to print all ternary strings of length three, i.e. all strings of length n, where each character belongs to $\{0, 1, 2\}$.

(b) Modify the program more generally as follows: declare a character array called symbols[], and generate all strings of length n, with each character in the string being from the symbols array.

6. A noun phrase is defined recursively as follows:

   - **Base Case:** Every noun is a noun phrase, eg: "car", "cat", "Alice", "building", etc.
   - **Recursion:** A string of the form: adjective noun phrase, is a noun phrase, eg: "fast red car", "tall building", etc.

   Write a function declared as void generateNounPhrase(char nounPhrase[], char commonNouns[],commonAdjectives[]), that generates a nounPhrase randomly as follows: with probability p, the function returns a noun; with probability $1 - p$ it returns a string of the form "adjective " concatenated with a noun phrase (recursive call).

   Use the files 50commonNouns.txt and 40commonAdjectives.txt and copy them yourself into arrays commonNouns[] and commonAdjectives[]. Use the strcat function from string.h for string concatenation.

7. (a) Manually create a file called 50MoreNouns.txt and add some 50 nouns into this file.

   (b) Write a program that reads the contents of the file 50CommonNouns.txt and copies it to another file called 100Nouns.txt.

   (c) Write a program that adds the contents of 50MoreNouns.txt to end of the file 100Nouns.txt.

8. **[Optional:]**Create a copy of redSanta.bmp in which the color of Santa's suit is changed from red to some other color of your choice.